

Guía de Aprendizaje — Pipeline CI/CD con GitHub Actions

Este documento explica **qué** construimos, **por qué** cada decisión, **qué alternativas** existían y sus **pros y contras**. Es el Proyecto 3 del portfolio: acá el protagonista ya no es el código de tests, sino la **orquestración** — cómo se ejecutan de forma automática, rápida y confiable.

Índice

1. Cómo usar esta guía
2. Qué es CI/CD y por qué es clave en QA
3. Anatomía de un workflow de GitHub Actions
4. La estrategia de dos velocidades
5. Quality gates: fallar rápido y barato
6. Job dependencies: etapas del pipeline
7. Smoke vs regresión: el rol de los tags
8. Concurrency: no malgastar minutos
9. Cron: correr solo cada noche
10. workflow_dispatch: disparo manual
11. Matriz y sharding: paralelismo horizontal
12. Reportes: blob y merge
13. Artifacts: guardar la evidencia
14. Notificaciones y secretos
15. Caché y npm ci vs npm install
16. UI + API en un mismo pipeline
17. Buenas prácticas y costos
18. Ejercicios para practicar
19. Glosario
20. Próximos pasos

1. Cómo usar esta guía

En los Proyectos 1 (UI) y 2 (API) construimos suites de tests. Pero una suite que hay que correr **a mano** en tu máquina aporta la mitad del valor. El salto de nivel es que corra **sola**, en cada cambio, en un servidor, actuando como red de seguridad antes de que el código llegue a producción. Eso es **CI/CD**, y es de lo más valorado en un perfil de automation senior.

Este proyecto trae una suite chica y combinada (UI + API, reutilizando lo aprendido) y le construye encima un **pipeline de nivel profesional** con **GitHub Actions**. El código de tests es el "material de trabajo"; lo que estudiamos acá es cómo se orquesta.

2. Qué es CI/CD y por qué es clave en QA

- **CI (Continuous Integration / Integración Continua):** cada cambio que un desarrollador integra al repositorio dispara automáticamente una batería de verificaciones (build, lint, tipos, tests). El objetivo es **detectar problemas apenas se introducen**, no días después.
- **CD (Continuous Delivery / Deployment):** llevar automáticamente ese código ya verificado hacia los ambientes (staging, producción). En este proyecto nos enfocamos en la parte de **CI** (la que más toca a QA), aunque las mismas herramientas hacen CD.

Por qué le importa tanto a QA

Sin CI, tu suite depende de que alguien se acuerde de correrla. Con CI:

- **Ningún cambio llega a main sin pasar por los tests.** El pipeline es un portero automático.
- **El feedback es inmediato:** el desarrollador se entera de que rompió algo en minutos, cuando el contexto está fresco y arreglarlo es barato.
- **La calidad se vuelve responsabilidad de todo el equipo**, no una etapa manual al final.

Frase para la entrevista: "automatizar tests sin CI es media automatización". El valor de una suite se multiplica cuando corre sola en cada cambio. Un QA Sr no solo escribe tests: los integra al flujo de desarrollo.

3. Anatomía de un workflow de GitHub Actions

GitHub Actions es la plataforma de CI/CD integrada en GitHub. Un **workflow** es un archivo YAML en `.github/workflows/` que describe qué hacer y cuándo. Sus piezas:

```

name: PR Checks          # nombre visible en la pestaña Actions

on:                      # DISPARADORES: cuándo corre
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:                    # los TRABAJOS (corren en paralelo salvo que se encadenen)
  quality-gates:         # un job
    runs-on: ubuntu-latest # el RUNNER: una máquina virtual limpia
    steps:               # los PASOS del job (en orden)
      - uses: actions/checkout@v4    # una "action" reutilizable
      - run: npm ci                  # un comando de shell

```

Conceptos clave:

- **on** — los eventos que disparan el workflow: `push`, `pull_request`, `schedule` (cron), `workflow_dispatch` (manual), etc.
- **jobs** — unidades de trabajo. Por defecto corren **en paralelo**; con `needs` se encadenan.
- **runs-on** — el runner: una máquina virtual efímera (Ubuntu, Windows o macOS) que GitHub crea limpia para cada job y destruye al terminar. **Cada job arranca de cero.**
- **steps** — los pasos dentro de un job. Pueden ser `uses:` (una *action* reutilizable de la comunidad) o `run:` (un comando).
- **Actions reutilizables** — como `actions/checkout` (traer el código) o `actions/setup-node` (instalar Node). Son bloques listos que no reinventás.

Dato clave que explica muchas decisiones: como cada job corre en una máquina **limpia y separada**, dos jobs **no comparten archivos** automáticamente. Si un job produce algo que otro necesita (un reporte), hay que pasarlo explícitamente con **artifacts** (sección 13). Por eso el merge de reportes es su propio job que descarga lo que produjeron los demás.

4. La estrategia de dos velocidades

La decisión de arquitectura más importante de este pipeline: **no todo corre en el mismo momento**. Tenemos dos workflows con propósitos opuestos:

	<code>pr-checks.yml</code>	<code>nightly-regression.yml</code>
Cuándo	En cada PR y push a main	Cada noche (cron) + a demanda
Qué corre	Solo <code>@smoke</code> , Chromium + API	TODO, los 3 browsers + API
Velocidad	Segundos	Minutos
¿Bloquea?	Sí (frena el merge)	No (informa)
Objetivo	Feedback inmediato	Cobertura exhaustiva

¿Por qué separarlos? Por un equilibrio entre velocidad y cobertura:

- Si en cada PR corriéramos TODA la regresión cross-browser, el desarrollador esperaría 20 minutos por cada cambio. Terminaría odiando el proceso o buscando cómo saltarlo.
- Si nunca corriéramos la regresión completa, se nos escaparían bugs específicos de Firefox o Safari.

La solución: en el PR, **lo rápido y de mayor valor** (que bloquea); de noche, **lo exhaustivo** (que no molesta a nadie porque corre mientras todos duermen). Este patrón —a veces llamado "pirámide de ejecución"— es una de las señales más claras de madurez en CI.

5. Quality gates: fallar rápido y barato

Un **quality gate** es una verificación automática que el código debe pasar. En `pr-checks.yml` , el primer job son dos gates baratos:

```
- name: ESLint
  run: npm run lint
- name: TypeScript
  run: npm run typecheck
```

Por qué estos van primero, antes de los tests: son rapidísimos (segundos, sin navegador) y atrapan clases enteras de errores sin necesidad de ejecutar nada. Si el código no compila o tiene un error de lint, **no tiene sentido gastar minutos instalando navegadores y corriendo tests**. Fallamos rápido y barato.

- **ESLint** (análisis estático): detecta problemas sin ejecutar el código — variables sin usar, promesas sin `await` , imports rotos. Se configura en `eslint.config.mjs` (formato "flat config" de ESLint 9).
- **TypeScript** (`tsc --noEmit`): verifica que todos los tipos sean coherentes. El `--noEmit` significa "solo revisá, no generes archivos".

Principio general de CI: "fail fast". Ordená las etapas de la más rápida/barata a la más lenta/cara. La primera que falle corta el pipeline, ahorrando tiempo y dinero.

6. Job dependencies: etapas del pipeline

Los jobs, por defecto, corren en paralelo. Con `needs:` los encadenamos en **etapas**:

```
jobs:
  quality-gates:
    # ...
  smoke:
    needs: quality-gates    # smoke SÓLO corre si quality-gates pasó
    # ...
```

Esto crea un pipeline por etapas:

```
quality-gates  —(si pasa)→  smoke
(rápido)                      (un poco más lento)
```

Por qué: implementa el "fail fast" a nivel de jobs. Si el lint o los tipos fallan, el job `smoke` **ni siquiera arranca** — no gastamos el tiempo de instalar Chromium y correr tests sobre código que ya sabemos que está roto. En pipelines más grandes esto se extiende: gates → unit → integración → e2e, cada etapa más cara que la anterior.

7. Smoke vs regresión: el rol de los tags

¿Cómo hace el pipeline para correr "solo lo crítico" en el PR y "todo" de noche, con la misma suite? Con **tags** en los tests:

```
test('login exitoso lleva al inventario @smoke @regression', ...) // en PR y de noche
test('usuario bloqueado muestra error @regression', ...)           // solo de noche
```

Y `--grep` filtra por tag:

```
# PR: solo smoke, en Chromium + API
playwright test --grep @smoke --project=chromium --project=api

# Nightly: todo (sin filtro), todos los proyectos
playwright test
```

- `@smoke` — el subconjunto crítico: los flujos que, si se rompen, todo lo demás no importa (¿se puede loguear? ¿se puede crear una reserva?). Rápido y bloqueante.
- `@regression` — la cobertura completa, incluidos los casos de borde y negativos.

Un mismo test puede tener ambos tags (es crítico Y parte de la regresión). Esta clasificación por tags es lo que permite que **una sola suite sirva a dos estrategias de ejecución distintas**.

8. Concurrency: no malgastar minutos

```
concurrency:
  group: pr-checks-${{ github.ref }}
  cancel-in-progress: true
```

El problema que resuelve: si empujás 3 commits seguidos a un PR, sin esto se lanzarían 3 runs completos, y los 2 primeros ya no importan (solo cuenta el último commit). Con `cancel-in-progress`, cuando llega un push nuevo, GitHub **cancela el run anterior en curso** del mismo branch y arranca uno con el código más reciente.

Beneficio: ahorra minutos de CI (que se pagan) y da siempre el resultado del último commit, sin runs zombis corriendo sobre código viejo. `${{ github.ref }}` en el `group` hace que la cancelación sea **por branch** (dos PRs distintos no se cancelan entre sí).

9. Cron: correr solo cada noche

La regresión completa se agenda con `schedule`:

```
on:
  schedule:
    - cron: '0 3 * * *'    # 03:00 UTC, todos los días
```

La sintaxis **cron** tiene 5 campos: `minuto hora día-del-mes mes día-de-semana`. `0 3 * * *` = "minuto 0 de la hora 3, todos los días". Los `*` significan "cualquier valor".

Por qué de noche: la regresión completa es pesada (cross-browser, todos los tests). Correrla cuando nadie está trabajando significa que a la mañana el equipo llega con el resultado listo, sin haber ralentizado ningún PR durante el día. Es el lugar ideal para lo lento pero exhaustivo.

Ojo: los cron de GitHub Actions corren en **UTC** y pueden demorarse unos minutos en épocas de mucha carga. Para horarios exactos o críticos, hay que tenerlo en cuenta.

10. workflow_dispatch: disparo manual

```
on:
  workflow_dispatch:
```

Esto agrega un botón "**Run workflow**" en la pestaña Actions. Permite disparar la regresión **a demanda**, sin esperar a la noche: por ejemplo, antes de un release importante, o para revalidar tras arreglar algo.

Por qué importa: un pipeline no debería depender solo de eventos automáticos. Poder ejecutarlo manualmente da control. (Se le pueden agregar `inputs` para parametrizarlo, por ejemplo elegir qué navegadores correr.)

11. Matriz y sharding: paralelismo horizontal

La regresión completa podría tardar bastante si corriera secuencial. La aceleramos con **matriz + sharding**:

```
strategy:
  fail-fast: false
  matrix:
    shard: [1, 2, 3]
steps:
  - run: npx playwright test --reporter=blob --shard=${{ matrix.shard }}/3
```

- **Matriz (`matrix`):** GitHub lanza un job por cada valor. Con `shard: [1, 2, 3]` se crean **3 jobs en paralelo**, en 3 máquinas distintas.
- **Sharding (`--shard=1/3`):** le dice a Playwright "de todos los tests, corré solo el primer tercio". Cada job corre un tercio diferente. Entre los 3, cubren el 100%, pero **en un tercio del tiempo**.
- **`fail-fast: false`** : si un shard falla, los otros **siguen**. Queremos ver **TODOS** los fallos de la regresión, no cortar al primero.

La diferencia con el paralelismo interno de Playwright: Playwright ya corre tests en paralelo *dentro* de una máquina (los workers). El sharding agrega paralelismo *entre* máquinas. Se combinan: 3 máquinas x varios workers cada una. Así una regresión que tardaría 15 minutos puede bajar a 5.

12. Reportes: blob y merge

Acá aparece un problema interesante del sharding: si 3 jobs separados corren cada uno un tercio de los tests, **cada uno genera su propio reporte parcial**. El usuario quiere UN reporte con todo, no tres.

La solución de Playwright son los **blob reports**:

```
# En cada shard: generar un reporte 'blob' (formato intermedio, mergeable)
- run: npx playwright test --reporter=blob --shard=${{ matrix.shard }}/3
- uses: actions/upload-artifact@v4          # subirlo como artifact
  with: { name: blob-report-${{ matrix.shard }}, path: blob-report/ }
```

Y un job aparte los une:

```
merge-report:
  needs: [regression]
  steps:
    - uses: actions/download-artifact@v4    # bajar los 3 blobs
      with: { pattern: blob-report-*, merge-multiple: true, path: all-blob-reports }
    - run: npx playwright merge-reports --reporter=html ./all-blob-reports
    - uses: actions/upload-artifact@v4      # subir el HTML combinado
```

El flujo: cada shard produce un **blob** (un formato binario intermedio, no visible directamente) → se suben como artifacts → un job final los **descarga todos y los mergea** en un único reporte HTML navegable.

Por qué **merge-report** es un job separado con **needs: [regression]**: recordá que cada job corre en una máquina distinta que no comparte archivos. El merge tiene que esperar a que los 3 shards terminen y **descargar** lo que produjeron. Este patrón (fan-out a shards → fan-in a un merge) es CI de nivel avanzado, muy valorado.

13. Artifacts: guardar la evidencia

Un **artifact** es un archivo o carpeta que un job produce y GitHub guarda para descargar después (o para que otro job lo consuma):

```
- uses: actions/upload-artifact@v4
  if: ${{ !cancelled() }}    # subilo incluso si el test falló
  with:
    name: html-report
    path: playwright-report/
    retention-days: 14       # cuánto tiempo se guarda
```

Por qué **if: \${{ !cancelled() }}**: queremos el reporte **especialmente cuando algo falla** (es cuando más lo necesitamos para diagnosticar). Sin ese **if**, un test rojo cortarí el job antes de subir el reporte. Con él, el reporte se sube pase lo que pase (salvo cancelación).

Los artifacts son **cómo dos jobs se pasan archivos** (upload en uno, download en otro) y **cómo el humano accede a la evidencia** (reportes, screenshots, videos, trases) después del run.

14. Notificaciones y secretos

El job `notify` avisa si la regresión falló:

```
notify:
  needs: [regression]
  if: failure()           # solo corre si algún shard falló
  steps:
    - run: echo "### ❌ La regresión falló" >> "$GITHUB_STEP_SUMMARY"
```

- `if: failure()` — este job solo se ejecuta si un job del que depende falló. Hay condiciones hermanas: `success()`, `always()`, `cancelled()`.
- `$GITHUB_STEP_SUMMARY` — un archivo especial: lo que escribís ahí aparece como resumen en la página del run. Notificación "gratis" dentro de GitHub.

Notificación real y secretos. En un proyecto real, acá avisarías a Slack, Teams o email. Eso requiere una credencial (un webhook URL) que **nunca** va en el código. Va en un **secret** del repo:

```
env:
  SLACK_WEBHOOK_URL: ${ secrets.SLACK_WEBHOOK_URL }
```

Los **secrets** de GitHub son variables cifradas (Settings → Secrets) que los workflows leen con `${ secrets.NOMBRE }` pero que nunca se muestran en los logs. Es el mismo principio que en los Proyectos 1 y 2: **credenciales fuera del repo, siempre**. En este proyecto dejamos el ejemplo de Slack comentado, porque no hay un webhook real configurado (y ser honesto sobre eso es parte del criterio).

15. Caché y npm ci vs npm install

Dos optimizaciones y una decisión importante:

Caché de dependencias:

```
- uses: actions/setup-node@v4
  with:
    node-version: 20
    cache: 'npm'          # cachea ~/.npm entre runs
```

Descargar todas las dependencias en cada run es lento. El `cache: 'npm'` guarda la caché de npm entre ejecuciones, acelerando el `npm ci`.

npm ci en vez de npm install: en los workflows usamos `npm ci`. La diferencia:

- `npm install` puede actualizar versiones y modificar el `package-lock.json`.

- `npm ci` instala **exactamente** las versiones del `package-lock.json`, de forma limpia y reproducible. Es más rápido y garantiza que el CI use las mismas versiones que vos localmente.

En CI siempre se usa `npm ci`: la **reproducibilidad** es sagrada. Que el pipeline instale algo distinto a lo que probaste sería una fuente de bugs fantasma.

16. UI + API en un mismo pipeline

Este proyecto orquesta tests de UI (con navegador) y de API (HTTP puro) en la misma suite. Se resuelve con los **proyectos** de Playwright, separados por `testMatch`:

```
projects: [  
  { name: 'chromium', testMatch: /ui\/.*\.spec\.ts/, use: { baseURL: ENV.uiBaseURL, ... } },  
  { name: 'api',      testMatch: /api\/.*\.spec\.ts/, use: { baseURL: ENV.apiBaseURL, ... } },  
]
```

- Los proyectos de navegador (`chromium`, `firefox`, `webkit`) solo toman los tests de `tests/ui/` y usan la baseURL de la UI.
- El proyecto `api` solo toma los de `tests/api/` y usa la baseURL de la API, con headers HTTP.

Ventaja para el pipeline: con `--project` elegimos qué correr en cada etapa. El PR corre `--project=chromium --project=api` (rápido); la nocturna corre todo. Un mismo repo, una config, y el pipeline decide la combinación según la velocidad que necesita. Además, cada proyecto tiene su propia `baseURL`, así que apuntar UI y API a ambientes distintos es trivial.

17. Buenas prácticas y costos

Resumen de las decisiones y por qué:

- **Fail fast:** gates baratos primero, tests caros después. La primera falla corta.
- **Dos velocidades:** rápido/bloqueante en PR, exhaustivo/informativo de noche.
- **Reproducibilidad:** `npm ci`, versiones pinneadas, runners limpios.
- **Concurrency:** cancelar runs viejos para no pagar de más.
- **Instalar solo lo necesario:** el smoke instala solo Chromium; la nocturna, todos los browsers. Los tests de API no instalan ninguno.
- **Secretos fuera del código:** siempre en GitHub Secrets.
- **Evidencia siempre:** artifacts con `if: !cancelled()`.
- **Resiliencia de red:** los pasos que dependen de la red (`npm ci`, `playwright install`) se reintentan, porque la infraestructura falla a veces.

Caso real de este proyecto: la primera corrida de la regresión nocturna falló, pero no por un bug: un shard tuvo un `ECONNRESET` (error de red transitorio) al hacer `npm ci`, mientras los otros dos pasaron. La lección es doble: (1) **no todo fallo en rojo es un bug de tu código** —a veces es la infraestructura—, y (2) un pipeline maduro es **resiliente** a eso. Por eso esos pasos ahora reintentan: `npm ci || (sleep 10 && npm ci) || (sleep 20 && npm ci)`. Distinguir un fallo de infraestructura de un fallo real es una habilidad central de un QA que trabaja con CI.

Sobre costos: GitHub Actions cobra por minuto de runner (con una cuota gratis generosa en repos privados). Todo lo de arriba —fail fast, concurrency, instalar solo lo necesario, sharding— además de dar mejor feedback, **reduce el costo**. Un QA Sr piensa también en la eficiencia del pipeline, no solo en que "pase".

18. Ejercicios para practicar

1. **Rompé un test a propósito** y abrí un PR: mirá cómo PR Checks se pone en rojo y bloquea el merge. Después arreglalo y velo pasar a verde.
2. **Agregá un input a** `workflow_dispatch` para elegir cuántos shards usar, y usalo en la matriz.
3. **Agregá un job de** `deploy` (simulado) que dependa del smoke con `needs`, para practicar la parte "CD".
4. **Configurá un secret** (por ejemplo `SLACK_WEBHOOK_URL` de prueba) y activá la notificación a Slack.
5. **Cambiá el cron** para que corra dos veces al día e interpretá la sintaxis.
6. **Agregá un badge** de un tercer estado al README y entendé cómo se arma la URL del badge.

19. Glosario

- **CI/CD:** Integración/Entrega Continua; automatizar verificación e integración de cada cambio.
- **GitHub Actions:** la plataforma de CI/CD integrada en GitHub.
- **Workflow:** archivo YAML que define qué automatizar y cuándo.
- **Job:** unidad de trabajo de un workflow; corre en su propia máquina.
- **Step:** paso dentro de un job (`run:` un comando, o `uses:` una action).
- **Runner:** la máquina virtual efímera donde corre un job.
- **Action:** bloque reutilizable (ej: `actions/checkout`).
- **Trigger (`on`):** el evento que dispara el workflow (push, PR, cron, manual).
- `needs` : dependencia entre jobs; crea etapas.
- **Quality gate:** verificación automática que el código debe pasar (lint, tipos).
- **Fail fast:** ordenar de barato a caro para cortar ante la primera falla.

- **Matriz (`matrix`)**: correr un job con varias combinaciones en paralelo.
- **Sharding**: partir la suite en fracciones que corren en máquinas distintas.
- **Blob report**: reporte parcial mergeable que produce cada shard.
- **Artifact**: archivo que un job guarda para descargar o pasar a otro job.
- **Concurrency**: control para cancelar runs redundantes del mismo branch.
- **Cron**: sintaxis para agendar ejecuciones por tiempo.
- `workflow_dispatch` : disparo manual desde la UI.
- **Secret**: credencial cifrada del repo, léida con `${{ secrets.X }}` .
- `npm ci` : instalación limpia y reproducible desde el lock file.

20. Próximos pasos

Con los Proyectos 1, 2 y 3 ya tenés la historia completa: **escribís tests (UI + API) y los orquestás en un pipeline profesional**. Eso es exactamente lo que hace un QA Automation Sr en el día a día. Lo que sigue:

- **Proyecto 4 — Estabilidad y flakiness**: el tema más senior. Este pipeline es la base perfecta: crear inestabilidad a propósito, verla fallar en CI, medir la tasa de flakiness y eliminarla. Acá se conectan los reintentos, el trace y las métricas de salud de la suite.
- **Proyecto 5 — Visual regression + contract testing (Pact)**: capas avanzadas que se integran a este mismo pipeline.

Cómo contarlo en una entrevista (problema → decisión → resultado): "Construí un pipeline de CI/CD en GitHub Actions para una suite de UI + API (decisión). El objetivo era feedback rápido en cada PR sin sacrificar cobertura (problema). Lo resolví con una estrategia de dos velocidades: quality gates + smoke bloqueante en el PR (segundos), y una regresión nocturna cross-browser con sharding en 3 y merge de reportes (resultado). El pipeline cancela runs redundantes, instala solo lo necesario en cada etapa y notifica ante fallos."

Y, como siempre: **lo construiste vos y entendés cada línea del YAML**. Eso es lo que te posiciona de verdad.